

CHƯƠNG 22: HỌC SÂU (DEEP LEARNING)

TRÍ TUỆ NHÂN TẠO - ARTIFICIAL INTELLIGENCE

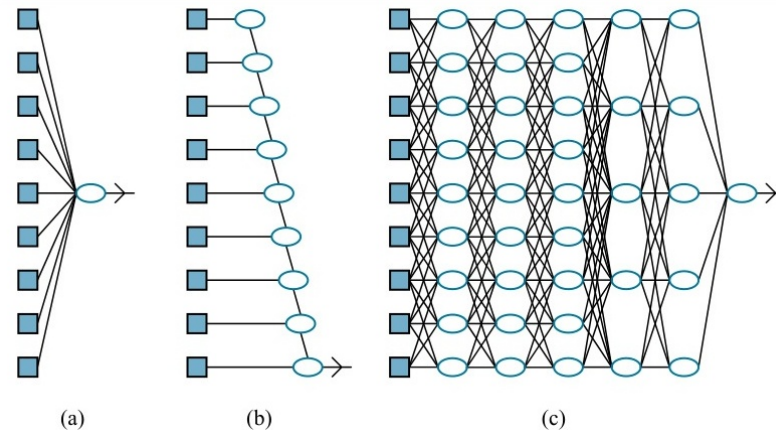
Nội dung Chương 22

- ◇ 22.1 Các Mạng Truyền Thẳng Đơn Giản (Feedforward)
- ◇ 22.2 Các Đồ Thị Tính Toán (Computation Graphs)
- ◇ 22.3 Các Mạng Tích Chập (Convolutional Networks - CNN)
- ◇ 22.4 Lan truyền ngược (Backpropagation)
- ◇ 22.5 Khái quát hóa & Dropout
- ◇ 22.6 Các Mạng Nơ-ron Hồi Quy (RNN & LSTM)
- ◇ 22.7 Học chuyển giao (Transfer Learning)

22.1 Các Mạng Truyền Thẳng (Feedforward)

◇ Lấy cảm hứng từ não bộ, **Mạng Nơ-ron Nhân tạo (ANN)** bao gồm các nơ-ron liên kết với nhau bằng các trọng số (Synapses).

◇ **Feedforward Neural Networks:** Dữ liệu chỉ chảy theo một chiều từ Tầng đầu vào (Input layer) → Tầng ẩn (Hidden layers) → Tầng đầu ra (Output layer).



◇ **Deep Learning (Học Sâu):** Là mạng Nơ-ron có **Nhiều tầng ẩn** (Từ vài chục đến hàng trăm tầng).

22.1 Hàm kích hoạt (Activation Functions)

◇ Nếu chỉ nhân trọng số $\mathbf{w}$ và cộng thiên kiến b , mạng Nơ-ron dù sâu 1000 tầng cũng chỉ tương đương với 1 phương trình tuyến tính duy nhất.

◇ **Hàm kích hoạt Phi tuyến (Non-linear Activation):** Đóng vai trò bẻ cong không gian, giúp AI học được các hàm phức tạp.

- **Sigmoid/Tanh:** Chặn output vào khoảng $[0, 1]$ hoặc $[-1, 1]$. Dễ bị *Triệt tiêu Gradient* ở mạng quá sâu.
- **ReLU (Rectified Linear Unit):** $f(x) = \max(0, x)$. Đơn giản, tính toán cực nhanh và giải quyết được Triệt tiêu Gradient. Là tiêu chuẩn vàng hiện nay.

22.2 Đồ Thị Tính Toán (Computation Graphs)

◇ Học Sâu hiện đại xem mạng Nơ-ron như một **Đồ thị tính toán có hướng (DAG)**.

- Các nút (Nodes) là các Phép toán Tensor (Cộng, Nhân Ma trận).
- Các cạnh (Edges) truyền dữ liệu là các Tensor (Mảng đa chiều).

◇ Nhờ biểu diễn dưới dạng Đồ thị Tính toán, các Framework như TensorFlow, PyTorch có thể ****Tự động tính đạo hàm (AutoGrad)**** mà lập trình viên không cần viết code giải đạo hàm bằng tay.

22.2 Hàm Mất mát (Loss Functions)

- ◇ Lớp cuối cùng (Output layer) quyết định cách AI xuất kết quả.
- ◇ **Hồi quy (Regression):** Nút ra không có hàm kích hoạt. Hàm mất mát thường là ****MSE**** (Mean Squared Error).
- ◇ **Phân loại Đa lớp (Multi-class Classification):**
 - **Softmax:** Biến đổi vector output thành một Phân bố xác suất (Tổng các phần tử = 1).
 - **Cross-Entropy Loss:** Đo lường sự khác biệt giữa Phân bố xác suất dự đoán và Nhãn thực tế (One-hot vector). $L = - \sum y \log(\hat{y})$.

22.3 Mạng Nơ-ron Tích Chập (CNN)

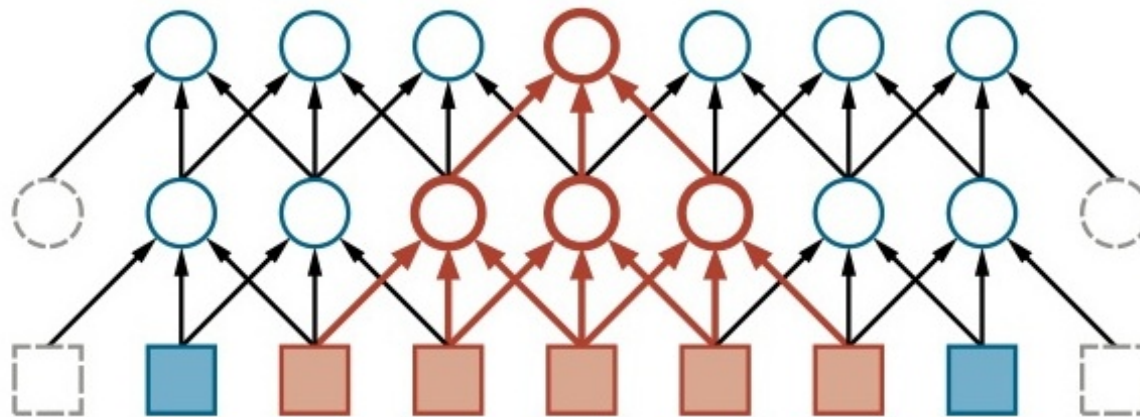
◇ **Vấn đề của Feedforward:** Nếu đầu vào là một ảnh 1000×1000 pixel, thì tầng ẩn đầu tiên cần *hàng tỷ trọng số*. Rất dễ Overfitting và tốn RAM bộ nhớ.

◇ **Convolutional Neural Networks (CNN):**

- Thay vì kết nối toàn bộ (Fully connected), CNN sử dụng các **Bộ lọc cục bộ (Local Filters / Kernels)** trượt trên bức ảnh.
- Tận dụng tính không gian của ảnh: Các pixel gần nhau thường có liên quan logic với nhau.
- **Chia sẻ trọng số (Weight sharing):** Một bộ lọc phát hiện "Đường chéo" có thể dùng chung cho mọi vị trí trên ảnh.

22.3 Hoạt động của Bộ lọc (Filters)

◇ Phép Tích chập (Convolution) là việc nhân chập ma trận bộ lọc nhỏ (VD: 3×3) với từng vùng của hình ảnh gốc.



◇ **Tính phân cấp phân cực:** Tầng 1 học *Cạnh*, *Góc*. Tầng 2 học *Mũi*, *Mắt*. Tầng cuối học tổng hợp toàn bộ *Khuôn mặt*.

22.3 Gộp và Mạng thặng dư (ResNet)

◇ Tầng Gộp (Pooling Layer):

- Thường dùng ****Max-Pooling****: Giữ lại giá trị lớn nhất trong vùng 2×2 .
- Giúp giảm một nửa kích thước dữ liệu (Downsampling), tiết kiệm RAM và tăng tính bất biến với dịch chuyển hình ảnh.

◇ Mạng Thặng Dư (ResNet):

- Khi mạng CNN sâu tới 100 tầng, thông tin đạo hàm truyền ngược bị triệt tiêu về 0.
- Giải pháp: Thêm ****Skip Connection**** (Đường tắt). Cộng trực tiếp đầu vào x của khối trước sang khối sau: $f(x) + x$.

22.4 Các Thuật Toán Học: Lan truyền ngược

◇ ****Lan truyền ngược (Backpropagation):**** Là trái tim của Deep Learning.

◇ Thuật toán tính đạo hàm của Hàm Loss theo từng tham số w của mạng từ lớp cuối lùi dần về lớp đầu, dựa trên **Quy tắc Dây chuyền (Chain Rule)** của vi tích phân:

$$\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial y_{out}} \times \frac{\partial y_{out}}{\partial h_j} \times \frac{\partial h_j}{\partial w_i}$$

◇ Kết hợp với ****Stochastic Gradient Descent (SGD)**** hoặc ****Adam Optimizer**** để cập nhật trọng số liên tục theo từng lô (Batch).

22.4 Chuẩn hóa theo lô (Batch Normalization)

◇ Trong quá trình học, do trọng số các tầng dưới thay đổi, phân bố dữ liệu ở tầng trên cũng bị xô lệch (Internal Covariate Shift), làm mạng học rất chậm.

◇ **Batch Normalization (BatchNorm):**

- Ép giá trị đầu ra của mỗi tầng ẩn trong một lô (mini-batch) trở về phân bố chuẩn (Trung bình = 0, Phương sai = 1).
- Cho phép sử dụng Tốc độ học (Learning rate) cực lớn → Học nhanh hơn gấp chục lần.

22.5 Tránh Khớp quá mức với Dropout

◇ Mạng Nơ-ron khổng lồ (hàng trăm triệu tham số) rất dễ học vẹt (Overfitting) dữ liệu Training.

◇ **Dropout:**

- Kỹ thuật cực kỳ sáng tạo: Trong quá trình Training, ****Tắt ngẫu nhiên**** một tỷ lệ p (VD: 50%) số Nơ-ron.
- Ép các Nơ-ron còn lại phải tự gánh vác việc suy luận mà không được ỷ lại vào Nơ-ron khác.
- Đóng vai trò như một cơ chế Ensemble (Bagging) ỏn khổng lồ bên trong một mạng duy nhất.

22.6 Mạng Nơ-ron Hồi Quy (RNN)

◇ Feedforward và CNN nhận đầu vào kích thước Cố định (1 bức ảnh).

◇ **Dữ liệu Chuỗi thời gian (Sequential):** Âm thanh, Giá cổ phiếu, Văn bản (Câu ngắn câu dài). Cần cấu trúc khác.

◇ **Recurrent Neural Networks (RNN):**

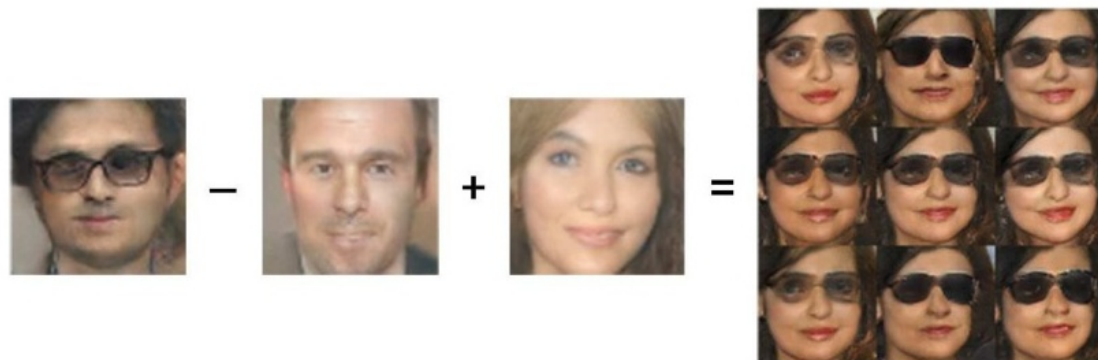
- Mạng có các **Vòng lặp (Loops)**. Trạng thái ẩn (Hidden state) của từ thứ t sẽ được truyền sang làm "Bộ nhớ" cho việc xử lý từ thứ $t + 1$.
- Vấn đề: Trí nhớ ngắn hạn. RNN cơ bản quên mất từ đầu câu khi đọc đến cuối câu.

22.6 Mạng LSTM (Long Short-Term Memory)

◇ ****LSTM**** là một biến thể tối tân của RNN giải quyết triệt để lỗi "Mất trí nhớ dài hạn".

◇ **Cơ chế các Cổng (Gates):**

- Thay vì ghi đè bộ nhớ liên tục, LSTM có Cổng Quên (Forget Gate), Cổng Cập nhật (Input Gate) và Cổng Đầu ra (Output Gate).
- Tế bào bộ nhớ (Cell state) chạy xuyên suốt chuỗi giống như băng chuyền, cho phép gradient truyền ngược đi rất xa mà không bị triệt tiêu.



22.7 Học Không Giám Sát và GANs

◇ Bộ Tự mã hóa (Autoencoders):

- Mạng Nơ-ron ép dữ liệu nén qua một "Nút thắt cổ chai" (Bottleneck) rồi cố gắng giải nén để phục hồi lại dữ liệu gốc.
- Mục đích: Tự động học các Đặc trưng nén (Latent Space) tốt nhất.

◇ GANs (Generative Adversarial Networks):

- Gồm 2 mạng đối đầu: Generator (Kẻ làm giả) cố tạo ảnh giả mạo để lừa discriminator, Discriminator (Cảnh sát) cố phân biệt ảnh thật/giả.
- Trận chiến này ép Generator phải sinh ra những bức ảnh (khuôn mặt người) chân thực đến mức mắt người cũng không nhận ra.

22.7 Học Chuyển Giao (Transfer Learning)

◇ Huấn luyện một mạng ResNet sâu 152 tầng từ con số 0 tốn hàng triệu USD tiền điện và thẻ GPU.

◇ **Học Chuyển Giao (Transfer Learning):**

- Tải xuống một mạng đã được Google/Facebook huấn luyện sẵn trên hàng triệu ảnh (Pre-trained Model).
- Đóng băng (Freeze) toàn bộ các tầng dưới (đã biết cách nhìn góc, cạnh).
- Chỉ huấn luyện lại (Fine-tune) tầng cuối cùng cho tập dữ liệu chuyên biệt của mình (Ví dụ: Phân loại Ảnh y tế).
- Hiệu quả đỉnh cao ngay cả khi chỉ có tập dữ liệu cực nhỏ.

Tóm tắt Chương 22

- ◇ **Backpropagation & AutoGrad:** Công cụ lan truyền ngược tự động giúp giải bài toán tối ưu hàng triệu biến.
- ◇ **CNN (Mạng Tích Chập):** Thống trị tuyệt đối lĩnh vực Thị giác máy tính nhờ chia sẻ trọng số và khai thác tính không gian cục bộ.
- ◇ **RNN / LSTM:** Khắc tinh của dữ liệu chuỗi (Thời gian, Âm thanh, Văn bản).
- ◇ **Kỷ nguyên AI hiện đại** hoàn toàn dựa trên Deep Learning với sự bùng nổ của cấu trúc mạng (ResNet, Transformers) và sức mạnh tính toán khổng lồ.